

CS 382 - Programming Assignment 2

Implementing Reliable Transport

February 25, 2026

Lead TAs: Ali Azhar, Dua Tariq, Mohammad Huzaifa, Muhammad, Shahab ud din
Due Date: March 23, 2026

Contents

1	Assignment Overview	2
2	Queries	2
3	Grade Distribution	2
4	Submission Instructions	2
5	Code Quality	2
5.1	Criteria	3
6	Introduction	3
6.1	Reliability Issues and Remedies	3
7	Provided Code	3
7.1	Everything at a glance	3
7.2	ReliableMessageSender	3
7.3	ReliableMessageReceiver	3
8	Implementation Instructions	4
8.1	Packets	4
8.1.1	Packet format	4
8.2	Reliable Transport Protocol	4
8.2.1	Sender Logic	4
8.2.2	Receiver Logic	4
9	Testing	5
10	Starter Code	5
11	Running the Code on Docker	5
11.1	Build and Run Container	5
11.2	Access Code	6
12	Tips on Getting Started	6
13	Academic Integrity	6

1 Assignment Overview

In this assignment, you will implement a **reliable transport protocol** on top of UDP. You will write logic for a message sender and a message receiver that communicate over UDP (an unreliable protocol by default), but **reliably** (which is your task).

Note

Implementation must be done **individually** and in **Python**. Your code will be tested in Docker.

2 Queries

Post any assignment-related queries **only on Slack**. Please avoid emailing the course staff with assignment-related queries so everyone benefits from responses.

3 Grade Distribution

This assignment is worth **12%** of your final grade and will be graded out of **35 points**.

- **BasicFunctionalityTest** (5): Connections are correctly set up, and data messages can be exchanged.
- **PacketLossTest** (5): Packets are reliably received despite losses inside the network.
- **OutOfOrderPacketsTest** (5): Packets are received in order by the end host even when reordered.
- **DuplicatePacketsTest** (5): Code correctly handles duplicate packets.
- **WindowSizeTest** (10): Correct window management and resilience against corruption.
 - Must follow **Selective Repeat** only.
 - Less rigorous version: 5 points; both less and more rigorous: 10 points.
 - Manual checking ensures conformance to guidelines; non-conformance yields **0** for this test.
 - Test cases run 3 times; you receive the average across runs.
- **Code Quality** (5): Evaluated by automated tools and manual review.

Important Warning

Assignments are tested in Docker (Ubuntu). “It works on my machine” is not a valid response.

4 Submission Instructions

Submit your assignment via **LMS** before the deadline.

Submission Format

- Submit a single zip file.
- The file must be named `<RollNumber>_PA2.zip`.
- The zip must contain the complete project directory.
- Only `reliable_transport.py` should be modified.

Important Warning

Failure to follow submission instructions will result in a 5% penalty.

5 Code Quality

Your code will be evaluated using automated tools and manual review.

5.1 Criteria

- **Maintainability, Modularity, Complexity:** assessed using tools like Radon and Pylint.
- **Naming Conventions (PEP 8):**
 - Variables / functions: `lowercase_with_underscores` (e.g., `calculate_sum()`).
 - Constants: `UPPERCASE_WITH_UNDERSCORES` (e.g., `MAX_SIZE`).
 - Classes: `CamelCase` (e.g., `ReliableSocket`).
- **Documentation / Comments:** must be sufficient for readability.
- **Modularity and Reusability:** avoid monolithic code; use helpers.
- **Readability / Formatting:** meaningful names, clean indentation and spacing.

6 Introduction

The network layer does not guarantee delivery of IP packets, so a reliable transport protocol is needed. You used such a protocol (TCP) in Programming Assignment 1. In this assignment, you will implement reliability mechanisms yourself.

6.1 Reliability Issues and Remedies

- **Packet Loss:** packets may be dropped in the network.
Remedy: receiver sends ACKs; sender retransmits on timeout.
- **Packet Corruption:** noise/interference can corrupt packets.
Remedy: checksum; receiver discards corrupted packets and does not ACK them.
- **Reordering:** packets may arrive out of order.
Remedy: sequence numbers; receiver reorders.
- **Duplicates:** retransmissions can cause duplicates.
Remedy: receiver ACKs duplicates but does not re-store or re-deliver.

For details, revisit lectures on reliable transport and congestion control.

7 Provided Code

This assignment builds on Programming Assignment 1. Instead of TCP sockets, `server.py` and `client.py` use `ReliableSocket` (in `reliable_socket.py`), which uses UDP.

7.1 Everything at a glance

- When `ReliableSocket` sends a message, it uses a `ReliableMessageSender` (in `reliable_transport.py`).
- For each incoming message, `ReliableSocket` initializes a `ReliableMessageReceiver` (in `reliable_transport.py`).

Your task is to implement `ReliableMessageSender` and `ReliableMessageReceiver` in `reliable_transport.py`. Do not modify any other file.

7.2 ReliableMessageSender

Implement `send_message` and `on_packet_received`. Use `self.send(packet)` to send packets.

- `__init__(..., window_size: int)`: set attributes; do not block.
- `send_message(message: str)`: may block until complete delivery.
- `on_packet_received(packet: str)`: invoked on incoming packets (typically ACKs); do not block.

7.3 ReliableMessageReceiver

Implement `on_packet_received`. Use `self.send(packet)` for ACKs and call `self.on_message_completed(message)` when the full message is received.

8 Implementation Instructions

8.1 Packets

You will use four packet types: **start**, **end**, **data**, and **ack**. All packets have a sequence number and checksum; data packets additionally carry message content.

8.1.1 Packet format

```
<packet_type>|<seq_num>|<msg_content>|<checksum>
```

Example start packet:

```
start|4212||1947410104
```

Example for message **request_users_list** with chunk size 10 bytes:

```
data|4213|request_us|2826007388
data|4214|ers_list|993936753
```

Use helpers from `util.py`:

```
data_packet = util.make_packet("data", 2, "request_us")
start_packet = util.make_packet("start", 1)
ack_packet = util.make_packet("ack", 2)
end_packet = util.make_packet("end", 3)

ok = util.validate_checksum(packet)
ptype, seq, content, cksum = util.parse_packet(packet)
```

8.2 Reliable Transport Protocol

A transmission begins with a **start** packet and ends with an **end** packet. Use a sliding window mechanism, specifically **Selective Repeat**.

For every received packet, the receiver sends a **cumulative ACK** indicating the **next expected** sequence number.

8.2.1 Sender Logic

1. Split the message into `util.CHUNK_SIZE` sized chunks.
2. Choose a random initial sequence number.
3. Reliably send a **start** packet (wait for ACK; retransmit if not received within `util.TIME_OUT`).
4. Send a window of **data** packets (window size is provided in the sender constructor) and process ACKs to slide the window.
5. If no ACKs are received for `util.TIME_OUT`, resend all packets in the current window.
6. After all chunks are delivered, reliably send an **end** packet.

Important Warning

Do **not** send **start** and **end** in the data window. Doing so will fail test cases.

8.2.2 Receiver Logic

1. Validate checksum; ignore corrupted packets.
2. Inspect `packet_type` and `sequence_number`.
3. On **start**: initialize receiver state; send ACK for `seq+1`.
4. On **data**: store packet if not a duplicate.
5. After storing, send cumulative ACK for (**highest contiguous seq received**) + 1.
6. On **end**: assemble chunks, call `self.on_message_completed(message)`, and ACK `seq+1`.

Important: Always ACK based on highest *contiguous* sequence received (no gaps), not the most recent packet.

9 Testing

To test locally:

```
python3 TestHarness.py
```

10 Starter Code

You have been provided a repository with (example) structure:

```
PA2/  
  docker-compose.yml  
  Dockerfile  
  README.md  
  requirements.txt  
  Code/  
    client.py  
    server.py  
    util.py  
    reliable_socket.py  
    reliable_transport.py  
    TestHarness.py  
    ReviewCodeQuality.py  
  Tests/  
    BasicTest.py  
    BasicFunctionalityTest.py  
    DuplicatePacketsTest.py  
    OutOfOrderPacketsTest.py  
    PacketLossTest.py  
    WindowSizeTest.py  
  Manual/  
    PA2.pdf
```

You will write code in:

- `reliable_transport.py` only.

11 Running the Code on Docker

A **Dockerfile** is included for consistent testing. Use the following steps:

11.1 Build and Run Container

When building and running the container for the first time, use the following commands:

```
1 docker compose build  
2 docker compose run --rm netcen-spring-2026
```

For all subsequent runs, use the following command:

```
1 docker compose run --rm netcen-spring-2026
```

What this command does:

- **run:** Executes the `netcen-spring-2026` service defined in the `docker-compose.yml` file as a temporary, one-off container.
- **-rm:** Automatically removes the container after the task completes, ensuring no leftover containers clutter the system.

- This command starts the container and runs the service or command specified for **netcen-spring-2026** in **docker-compose.yml**.

11.2 Access Code

All your local code files are accessible at **/home/netcen_pa2** inside the container. You can edit them from your host machine, and changes will be reflected in the container.

12 Tips on Getting Started

1. Start early. Read the handout carefully.
2. Use the helper functions in **util.py**.
3. Make sure your server code does not crash unless explicitly told to.
4. Test inside Docker at least once before submission.

13 Academic Integrity

Academic integrity is paramount. Any form of plagiarism, including but not limited to copying code from online source as well as unauthorized collaboration is prohibited.

All submissions may be analyzed using plagiarism detection tools. If plagiarism is detected, the case may be forwarded to the DC.

Good Luck, and Happy Coding!